

MATENCHI KAGISO

JUNE 2026

kagisomatenchi16@gmail.com / kagisokgrey@gmail.com

```
/* assessment_cover.sql */  
  
SELECT  knowledge, skills, results  
FROM    'Kagiso Matenchi'  
WHERE   topic = 'UNION & UNION ALL'  
AND     platform = 'Databricks';
```

UNION & UNION ALL

STUDENT ASSESSMENT · SQL ON DATABRICKS

OPERATOR 01

UNION

removes duplicates

OPERATOR 02

UNION ALL

keeps all rows

QUESTIONS

10 + 3 BONUS

beginner level

KAGISO MATENCHI

SQL PRACTITIONER · BRIGHTLEARN

▶ RUN ON DATABRICKS

-- end of cover page --

All twenty tables come from a single fictional African retail bank called NexBank.

AND THE SQL ENVIRONMENT USED IS ON DATABRICKS(SQL EDITOR)

ABOUT THE UNION AND UNION ALL:

In SQL, **UNION** and **UNION ALL** are used to combine the results of two or more SELECT queries into a single result set.

UNION combines the results of multiple queries and automatically removes duplicate rows. This means that if the same record appears in more than one query result, it will only appear once in the final output. It is useful when you want a distinct list of records.

UNION ALL, on the other hand, also combines the results of multiple queries, but it does not remove duplicates. All rows from each query are included in the final result, even if some are identical. Because it does not perform duplicate checking, **UNION ALL is generally faster than UNION**.

In summary, the main difference is that **UNION returns only unique records**, while **UNION ALL returns all records including duplicates**.

QUESTION 1 :

databricks Search data, notebooks, recents, and more... CTRL + P workspace

SQL Editor Focus: ON

Run selected (1000) 3 minutes ago (2) workspace, default New SQL editor: ON

```

239 SELECT
240   account_id,
241   account_holder,
242   CASE
243     WHEN account_id IN (SELECT account_id FROM branch_sandton_accounts)
244     AND account_id IN (SELECT account_id FROM branch_rosebank_accounts)
245     THEN 'Both'
246     WHEN account_id IN (SELECT account_id FROM branch_sandton_accounts)
247     THEN 'Sandton'
248     ELSE 'Rosebank'
249   END AS city
250 FROM (
251   SELECT account_id, account_holder FROM branch_sandton_accounts
252   UNION
253   SELECT account_id, account_holder FROM branch_rosebank_accounts
254 ) AS combined;

```

Add parameter

Table	account_id	account_holder	city
1	AA001	Lerato Khumalo	Sandton
2	A003	Lerato Sithole	Both
3	A004	Peter Nkosi	Both
4	A001	Nomvula Dlamini	Sandton
5	A006	Thabo Mtho	Rosebank
6	A005	Zanele Khumalo	Rosebank

6 rows | 1.95s runtime See performance (1) Checked for improvements Refreshed 3 minutes ago

QUESTION 2 :

databricks Search data, notebooks, recents, and more... CTRL + P workspace

SQL Editor Focus: ON

Run selected (1000) Just now (1) workspace, default New SQL editor: ON

```

256 --Q2 : TWO PRODUCT LIST
257 SELECT product_code,
258        product_name,
259        CASE
260          WHEN product_code IN (SELECT product_code FROM savings_products)
261          AND product_code IN (SELECT product_code FROM current_products)
262          THEN 'Falls underboth '
263          WHEN product_code IN (SELECT product_code FROM savings_products) THEN 'Savings'
264          ELSE 'Current'
265        END AS product_type
266 FROM (SELECT product_code, product_name
267       FROM savings_products
268       UNION
269       SELECT product_code, product_name
270       FROM current_products) AS combined;

```

Add parameter

Table	product_code	product_name	product_type
1	SV01	Basic Savings	Savings
2	SV02	Premium Savings	Savings
3	SV03	Youth Savings	Falls underboth
4	SV04	Business Savings	Savings
5	CR01	Standard Current	Current
6	CR03	Business Current	Current
7	CR02	Gold Current	Current

7 rows | 2.82s runtime See performance (1) Checked for improvements Refreshed now

QUESTION 3:

The screenshot shows the Databricks SQL Editor interface. The left sidebar contains navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Spaces, Alerts, Query History, SQL Warehouses, Data Engineering, Runs, and Data Ingestion. The main area displays a SQL query in the editor, which has been executed. The query is a SELECT statement with a CASE clause and a UNION. The results are shown in a table below the query.

```

273 SELECT staff_id,
274        staff_name,
275        CASE
276        WHEN staff_id IN (SELECT staff_id FROM retail_banking_staff)
277          AND staff_id IN (SELECT staff_id FROM corporate_banking_staff)
278          THEN 'Assigned to both'
279        WHEN staff_id IN (SELECT staff_id FROM retail_banking_staff)
280          THEN 'Retail'
281          ELSE 'Corporate'
282        END AS email
283 FROM (SELECT staff_id, staff_name
284       FROM retail_banking_staff
285       UNION
286       SELECT staff_id, staff_name
287       FROM corporate_banking_staff) AS combined;

```

staff_id	staff_name	email
S003	Aisha Patel	Assigned to bo...
S002	Brian Tshabalala	Retail
S001	Mpho Radebe	Retail
S004	Kabelo Moabelo	Assigned to bo...
S005	Nandi Dube	Corporate
S006	Sipho Khumalo	Corporate

6 rows | 0.18s runtime | See performance (1) | Checked for improvements | Refreshed now

QUESTION 4 :

The screenshot shows the Databricks SQL Editor interface. The left sidebar contains navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Spaces, Alerts, Query History, SQL Warehouses, Data Engineering, Runs, and Data Ingestion. The main area displays a SQL query in the editor, which has been executed. The query is a SELECT statement with a CASE clause and a UNION. The results are shown in a table below the query.

```

289 --Q4 : MARKETING
290 SELECT city_code,
291        city_name,
292        CASE
293        WHEN city_code IN (SELECT city_code FROM mobile_branch_cities) AND
294          city_code IN (SELECT city_code FROM digital_branch_cities) THEN 'Falls underboth'
295        WHEN city_code IN (SELECT city_code FROM mobile_branch_cities) THEN 'Mobile'
296        ELSE 'Digital'
297        END AS region
298 FROM (SELECT city_code, city_name
299       FROM mobile_branch_cities
300       UNION
301       SELECT city_code, city_name
302       FROM digital_branch_cities) AS combined;
303

```

city_code	city_name	region
C02	Pretoria	Mobile
C01	Johannesburg	Falls underboth
C03	Cape Town	Falls underboth
C04	Durban	Mobile
C06	Port Elizabeth	Digital
C05	Polokwane	Digital

6 rows | 1.83s runtime | See performance (1) | Checked for improvements | Refreshed now

QUESTION 5:

databricks Search data, notebooks, recents, and more... CTRL + P workspace

SQL Editor Focus: ON + X

Run selected (1000) Just now (0s) workspace: default New SQL editor: ON

```

304 --Q5 : DIGITAL MARKETING TEAM
305 SELECT customer_id,
306        customer_name,
307 CASE
308 WHEN customer_id IN (SELECT customer_id FROM push_notification_targets ) AND
309        customer_id IN (SELECT customer_id FROM inapp_banner_targets ) THEN 'Falls underboth'
310 WHEN customer_id IN (SELECT customer_id FROM push_notification_targets ) THEN 'Push Notification'
311 ELSE 'Inapp Banner'
312 END AS segment
313 FROM (SELECT customer_id, customer_name
314       FROM push_notification_targets
315       UNION
316       SELECT customer_id, customer_name
317       FROM inapp_banner_targets) AS combined;
318

```

Add parameter

Table	customer_id	customer_name	segment
1	C1003	Fatima Mahomed	Falls underboth
2	C1004	Ryno van Zyl	Push Notification
3	C1001	Nomsa Zwane	Push Notification
4	C1002	Andile Buthelezzi	Falls underboth
5	C1006	Samuel Nkosi	Inapp Banner
6	C1005	Thandeka Cele	Inapp Banner

6 rows | 1.71s runtime See performance (1) Checked for improvements Refreshed now

QUESTION 6:

databricks Search data, notebooks, recents, and more... CTRL + P workspace

SQL Editor Focus: ON + X

Run selected (1000) 2 minutes ago (2s) workspace: default New SQL editor: ON

```

320 SELECT *
321 FROM atm02_transactions;
322 --Q6 : FRAUD TEAM
323 SELECT
324     txn_id AS transaction_id, account_id, amount,
325     CURRENT_DATE AS transaction_date
326 FROM atm01_transactions
327 UNION ALL
328 SELECT
329     txn_id, account_id, amount,
330     CURRENT_DATE AS transaction_date
331 FROM atm02_transactions;
332

```

Add parameter

Table	transaction_id	account_id	amount	transaction_date
1	T1001	A001	500.00	2026-06-10
2	T1002	A002	1200.00	2026-06-10
3	T1003	A001	300.00	2026-06-10
4	T1004	A003	750.00	2026-06-10
5	T1003	A001	300.00	2026-06-10
6	T1005	A004	900.00	2026-06-10
7	T1006	A002	450.00	2026-06-10
8	T1007	A005	150.00	2026-06-10

8 rows | 1.86s runtime See performance (1) Checked for improvements Refreshed 2 minutes ago

QUESTION 7:

databricks Search data, notebooks, recents, and more... CTRL + P workspace

SQL Editor Catalog Focus: ON + X

Run selected (1000) Just now (1s) workspace default New SQL editor: ON

```

337
338
339 -- Q7: CREDIT TEAM
340 SELECT
341   app_id AS application_id, customer_id, loan_type,
342   0.00 AS amount_requested
343 FROM gauteng_loan_applications
344 UNION ALL
345 SELECT
346   app_id, customer_id, loan_type,
347   0.00 AS amount_requested
348 FROM western_cap_e_loan_applications;
349

```

Add parameter

Table	+			
application_id	customer_id	loan_type	amount_requested	
1	LA001	C1001	Home Loan	0.00
2	LA002	C1002	Vehicle Loan	0.00
3	LA003	C1003	Personal	0.00
4	LA004	C1004	Home Loan	0.00
5	LA003	C1003	Personal	0.00
6	LA005	C1005	Vehicle Loan	0.00
7	LA006	C1006	Business	0.00
8	LA007	C1007	Home Loan	0.00

8 rows | 1.41s runtime See performance (1) Checked for improvements Refreshed now

QUESTION 8:

databricks Search data, notebooks, recents, and more... CTRL + P workspace

SQL Editor Catalog Focus: ON + X

Run selected (1000) Just now (1s) workspace default New SQL editor: ON

```

355
356 -- Q8 : CUSTOMER COMPLAINTS
357 SELECT complaint_id, customer_id, CURRENT_DATE AS logged_date,
358 CASE
359   WHEN complaint_id IN (SELECT complaint_id FROM email_complaints) THEN 'Email'
360   ELSE 'App'
361 END AS category
362 FROM (SELECT complaint_id, customer_id
363       FROM email_complaints
364       UNION ALL
365       SELECT complaint_id, customer_id
366       FROM app_complaints) AS combined;
367

```

Add parameter

Table	+			
complaint_id	customer_id	logged_date	category	
1	EC001	C2001	2026-06-10	Email
2	EC002	C2002	2026-06-10	Email
3	EC003	C2003	2026-06-10	Email
4	EC004	C2004	2026-06-10	Email
5	AC001	C2005	2026-06-10	App
6	AC002	C2001	2026-06-10	App
7	AC003	C2006	2026-06-10	App
8	AC004	C2007	2026-06-10	App

8 rows | 1.11s runtime See performance (1) Checked for improvements Refreshed now

QUESTION 9:

databricks Search data, notebooks, recents, and more... CTRL + P workspace

SQL Editor Focus: ON

Run all (1000) 3 minutes ago (1) workspace default New SQL editor: ON

```
--Q9 : FINANCE
SELECT payment_id,account_id,amount,
CASE
WHEN payment_id IN (SELECT payment_id FROM april_payments) THEN 'In April'
ELSE 'In May'
END AS payment_date
FROM(SELECT payment_id,account_id,amount
FROM april_payments
UNION ALL
SELECT payment_id,account_id,amount
FROM may_payments) AS combined;
```

Add parameter

	payment_id	account_id	amount	payment_date
1	PAY001	A001	12500.00	In April
2	PAY002	A002	4800.00	In April
3	PAY003	A003	9200.00	In April
4	PAY004	A004	3300.00	In April
5	PAY005	A001	12500.00	In May
6	PAY006	A005	7600.00	In May
7	PAY007	A002	5100.00	In May
8	PAY008	A006	2800.00	In May

8 rows | 1.29s runtime See performance (1) Checked for improvements Refreshed 3 minutes ago

QUESTION 10:

databricks Search data, notebooks, recents, and more... CTRL + P workspace

SQL Editor Focus: ON

Run selected (1000) Just now (1) workspace default New SQL editor: ON

```
--Q10: ACCOUNTING SYSTEM
SELECT entry_id,account_id,entry_type,
0.00 AS amount, CURRENT_DATE-3 AS entry_date
FROM debit_entries
UNION ALL
SELECT entry_id,account_id,entry_type,
0.00 AS amount, CURRENT_DATE-2 AS entry_date
FROM credit_entries;
```

Add parameter

	entry_id	account_id	entry_type	amount	entry_date
1	DR001	A001	Debit	0.00	2026-06-07
2	DR002	A002	Debit	0.00	2026-06-07
3	DR003	A003	Debit	0.00	2026-06-07
4	DR004	A004	Debit	0.00	2026-06-07
5	CR001	A001	Credit	0.00	2026-06-08
6	CR002	A005	Credit	0.00	2026-06-08
7	CR003	A002	Credit	0.00	2026-06-08
8	CR004	A006	Credit	0.00	2026-06-08

8 rows | 1.15s runtime See performance (1) Checked for improvements Refreshed now

BONUS 1:

They should use UNION,

UNION automatically removes duplicate rows, which means that if a customer appears in both months, they will only appear once in the final result. In contrast, UNION ALL keeps all rows from both lists, including duplicates. As a result, the same customer may appear more than once, making it difficult to produce a clean list of unique customers.

BONUS 2:

They should use UNION ALL,

If UNION was used, any duplicate transactions recorded in both systems would be removed, which could lead to an inaccurate transaction count and potentially conceal recording errors. By using UNION ALL, all transactions from both systems are retained, allowing the auditing team to view the complete set of transactions recorded during Q1.

BONUS 3:

The issue is that the two SELECT statements return a different number of columns. The first SELECT returns two columns (account_id and account_holder), while the second returns three columns (account_id, account_holder, and city). This violates one of the basic rules of UNION, which requires both SELECT statements to return the same number of columns.

To fix the problem, both SELECT statements need to return the same number of columns. This can be done by adding a

city value to the first SELECT, so that both queries return three columns:

For instance

```
SELECT account_id, account_holder, 'Sandton' AS city  
FROM branch_sandton_accounts  
UNION  
SELECT account_id, account_holder, city  
FROM branch_rosebank_accounts;
```

By adding 'Sandton' AS city to the first query, both SELECT statements now return the columns account_id, account_holder, and city, satisfying the requirements of the UNION operation.